SYSTEM INTEGRATION REPORT

AI Student Success Recommendation Platform

Course	AIE1014 — AI Applied Project
Milestone	Milestone 4 — System Integration
Due Date	Wednesday, March 25, 2026 at 12:30 PM
Weight	10% of Final Grade
Team Name	Team Student Success
Stakeholder / Client	Cambrian College — Student Success & Retention Office
Date	March 18, 2026
GitHub Repository	https://github.com/Odunzek/ai-student-success-recommendation

Team Members

Name	Role	Contribution to This Milestone
Daniel Quartey	Project Lead	Led milestone planning, coordinated team deliverables, reviewed integration outputs, and ensured alignment with stakeholder requirements.
Odunze Oyekachi	Data & ML Engineer	Trained and validated the XGBoost classifier, integrated the SHAP explainer, and prepared the data pipeline for CSV ingestion.
Juthamard Jirapanyalerd	Backend & LLM Integration	Built FastAPI endpoints, integrated the OpenAI LLM provider for chat and interventions, and resolved SHAP/NumPy compatibility issues.
Ime-Jnr Ime-Essien	Frontend Engineer	Developed the React + TypeScript UI, integrated API hooks for predictions and student data, and implemented markdown rendering for chat replies.

Table of Contents

1. Executive Summary
2. API Design
3. User Interface Design
4. Integration Architecture
5. Testing
6. Deployment Instructions
7. Known Issues & Future Work
8. Appendix

1. Executive Summary

1.1 Project Overview

This project provides an AI-powered Student Success Platform that predicts which students are at risk of dropping out and recommends tailored interventions. It is designed for academic advisors and student success teams at Cambrian College to:

- Identify high-risk students early using machine learning predictions.
- Explain why a student is at risk via SHAP-based feature attributions.
- Support advisors with AI-generated intervention suggestions and an integrated chat assistant.

The integrated system combines a production-style backend API, a machine learning model trained on student performance and engagement data, and a modern web UI for exploration and action.

1.2 System Summary

Component	Technology	Status
Machine Learning Model	XGBoost Classifier + SHAP explainer	Complete
API	FastAPI + Uvicorn	Complete
User Interface	React + TypeScript + Tailwind CSS	Complete
Integration Tests	Python (pytest + requests)	Complete

1.3 Key Accomplishments

Accomplishment	Description
End-to-end prediction flow	User can select a student, call the prediction API, and see risk score + SHAP explanations.
Student context integration	UI surfaces real student records and a detailed student modal bound to /students API.
Dashboard & alerts	Dashboard shows high-risk counts, average predicted score, and alerts linked to profiles.
LLM-powered interventions	Chat and interventions use an LLM (OpenAI/Ollama) to generate personalised recommendations.

1.4 Demo URL

- API Docs (local): http://127.0.0.1:8000/docs
- Frontend (local dev): http://127.0.0.1:5173/

2. API Design

2.1 API Overview

Attribute	Value
Framework	FastAPI
Base URL	http://127.0.0.1:8000
API Prefix	/api/v1
Authentication	None (for local development)
Response Format	JSON

The API exposes endpoints for health checks, prediction, dashboard summaries, student records, file upload, chat, and settings.

2.2 Endpoints

GET / (Root Health Check)

Attribute	Value
Method	GET
Purpose	Basic health check and meta info
Auth	None

Response:

```
{ "status": "healthy", "api_version": "1.0.0", "model_loaded": true }
```

POST /api/v1/predict/student

Attribute	Value
Method	POST
Purpose	Predict dropout risk for a single student
Content-Type	application/json

Request Parameters:

Parameter	Type	Required	Description	Example
student_id	string	No	ID used to link back to UI	"STU004"
num_of_prev_attempts	int	Yes	Prior attempts for the module	1
studied_credits	int	Yes	Total credits enrolled	120
avg_score	float	Yes	Average assessment score	55.0
total_clicks	int	Yes	LMS clicks (engagement proxy)	180
completion_rate	float	Yes	Completed / total assessments	0.45
module_BBB...GGG	int	Yes	One-hot encoded module enrolment	0 or 1

Success Response (200):

```
{ "student_id": "STU004", "risk_score": 78, "risk_probability": 0.78,
  "risk_level": "high", "prediction": 1,
  "shap_factors": [{ "feature": "avg_score", "value": 55.0,
    "impact": 0.22, "direction": "positive" }, ...],
  "status": "success" }
```

Error Response (400):

```
{ "error": "Missing required field: avg_score", "status": "error" }
```

GET /api/v1/students/{student_id}

Returns the full student record used to populate the UI modal. Response includes demographics, academic data, engagement metrics, and grade history.

GET /api/v1/dashboard/summary

Returns aggregate counts and averages: total students, high-risk count, medium-risk count, and average dropout probability.

POST /api/v1/chat

Accepts a student_id and a natural-language message, then returns an LLM-generated recommendation reply along with the provider and model used.

2.3 Error Handling

Error Code	Meaning	When Returned
200	Success	Valid request processed
400	Bad Request	Missing or invalid input
404	Not Found	Student ID or endpoint not found
500	Server Error	Unexpected error, SHAP/model load failures

Errors are returned as JSON with error and status fields.

2.4 API Design Decisions

- Use RESTful JSON endpoints for simple integration with React and other clients.
- Use Pydantic models for strict input validation and clear error messages.
- Version API under /api/v1 for forward compatibility.
- Keep prediction, student records, dashboard, upload, settings, and chat in separate routers for maintainability.
- Expose AI intervention and chat as separate endpoints so the prediction core remains usable even when LLM is disabled.

3. User Interface Design

3.1 UI Overview

Attribute	Value
Framework	React 18 + TypeScript + Vite
Styling	Tailwind CSS
State Mgmt	Zustand + TanStack React Query
URL (dev)	http://127.0.0.1:5173/
Target User	Academic advisors / retention staff
Primary Use	Monitor risk, inspect students, plan support

3.2 UI Screens

Main Interface — Dashboard

Cards showing At-Risk Students, Interventions Active, Avg Predicted Score, and an AI Insight card; risk distribution chart and alerts table.

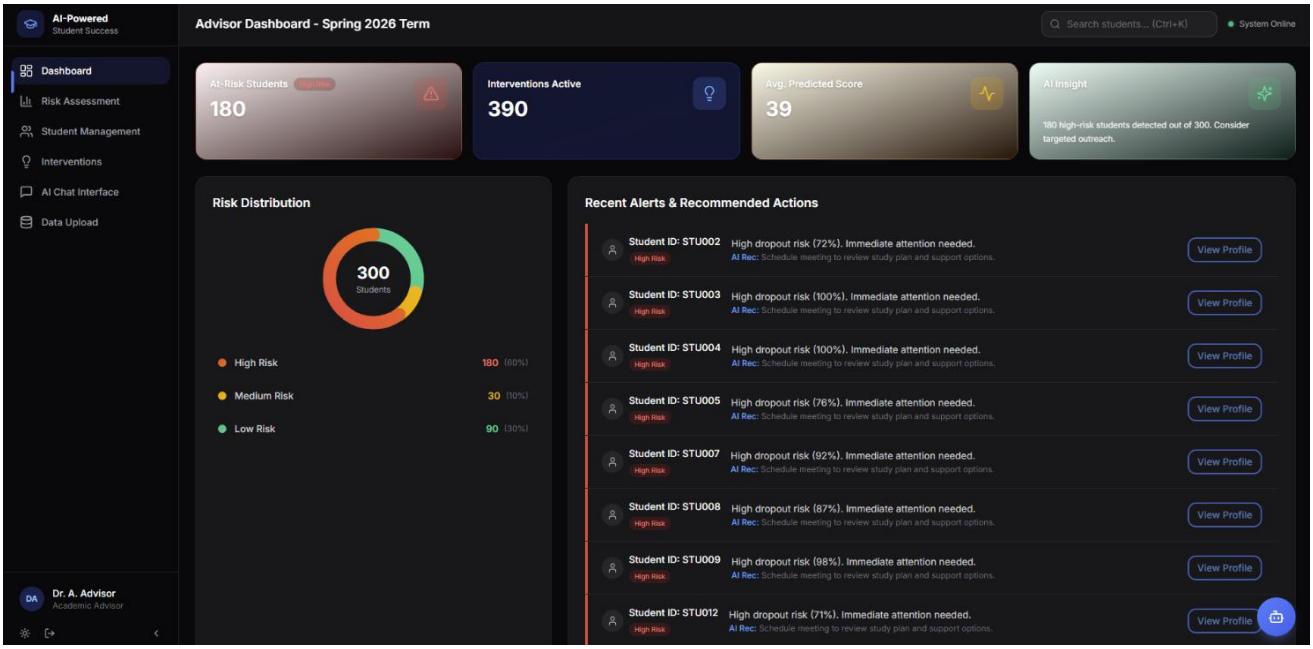

Figure 1: Dashboard overview

Prediction Results — Student Details Modal

Shows demographics, academic record, engagement metrics, grades, and formatted SHAP explanation text.

Figure 2: Student details modal with SHAP breakdown

Chat Interface

Conversation history with AI, student context selector, and markdown-rendered replies (bold headings + numbered lists).

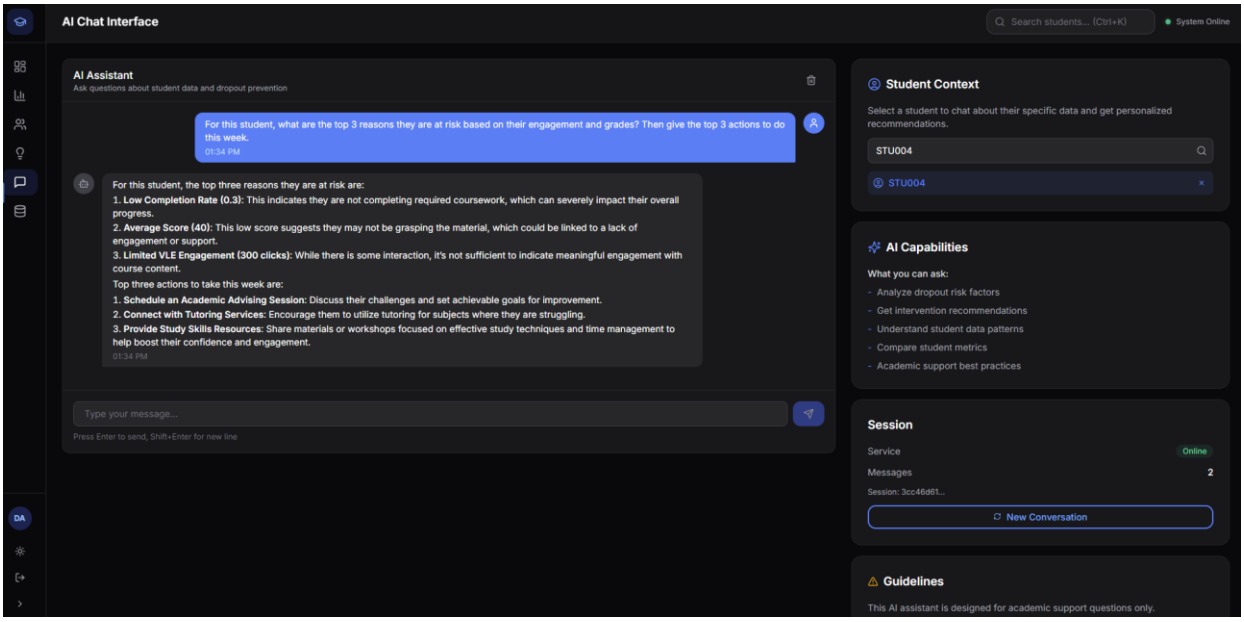

Figure 3: Chat interface with AI advisor

Error Handling

Student ID or endpoint not found (404: Not Found).

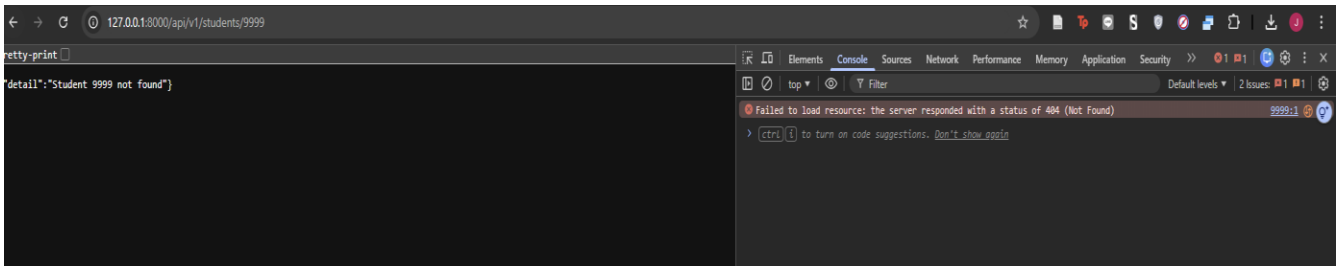

Figure 4: 404 error state

3.3 User Flow

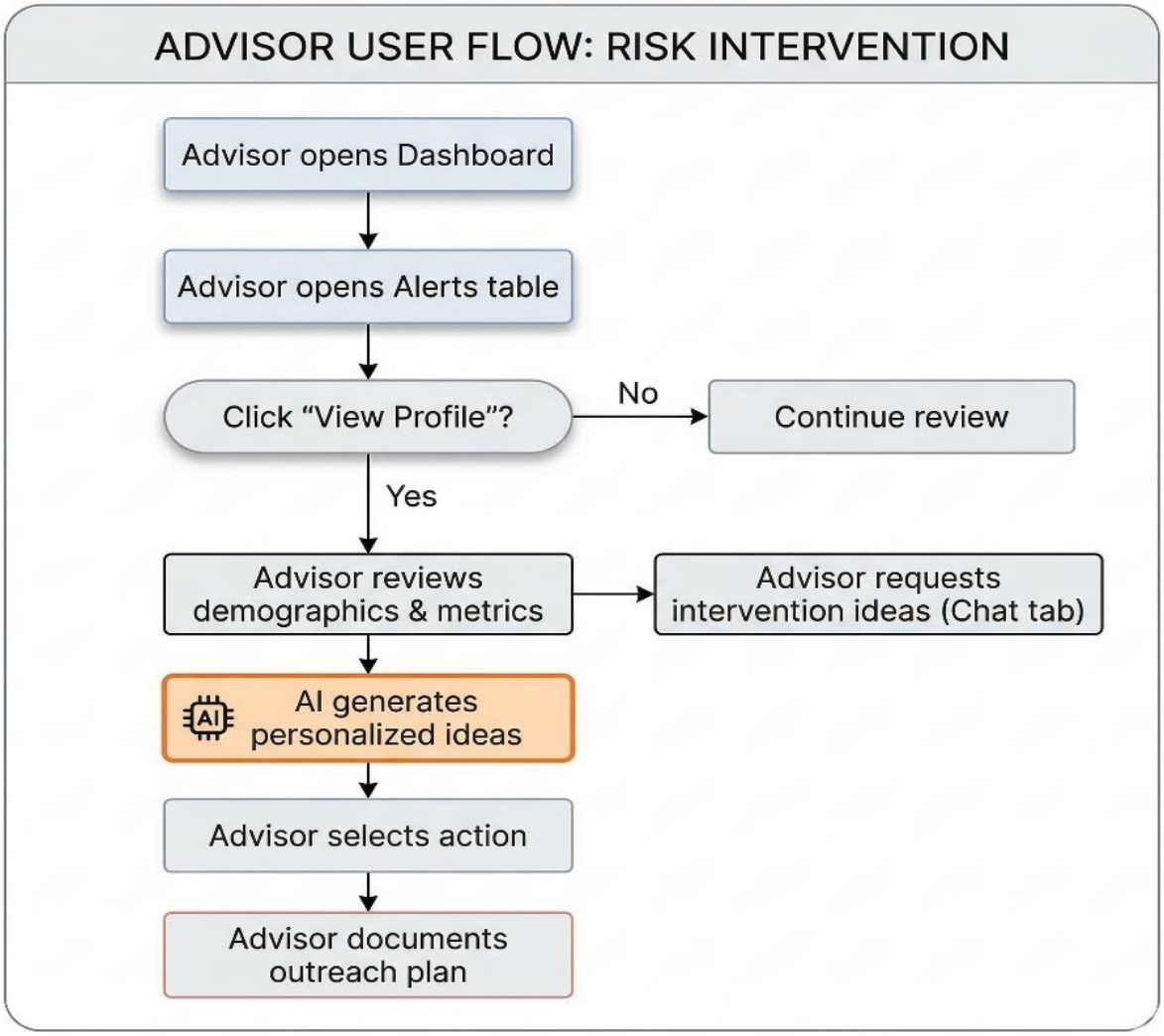

Figure 5: Advisor user flow diagram

- Advisor opens the dashboard and reviews high-level risk statistics and AI Insight.
- Advisor opens the alerts table and clicks “View Profile” to inspect specific high-risk students.
- In the student modal, advisor reviews demographics, engagement metrics, and explanation details.
- Advisor navigates to the Chat tab, selects a student, and requests personalised intervention ideas.
- Advisor uses suggestions to plan follow-up (meetings, tutoring, outreach).

3.4 Input Components

Input	UI Component	Validation
Student search	Text input + dropdown	Debounced search, optional
Risk filter	Filter chips (Low / Medium / High)	Client-side filtering only
Chat message	Textarea + Send button	Non-empty before send
File upload	File input for CSV	Checked server-side for schema
Prediction form	Number / select inputs	Ranges and type enforced (Pydantic)

3.5 Output Display

Output	Display Method	Notes
Risk summary	Stat cards with animated numbers and icons	High-contrast backgrounds for readability
Risk distribution	Chart component	Groups by low / medium / high
Student details	Modal with labelled sections	Displays “—” instead of “N/A”
SHAP explanations	List / text summary of top features	Shows feature, value, impact, direction
Chat replies	Chat bubbles rendered with markdown	Bold headings and numbered lists
Upload status	Toasts and badges	Shows latest upload and row count

3.6 UI Design Decisions

- Use a dashboard-first layout to give advisors a quick snapshot of risk.
- Use a modal instead of separate pages for student details to reduce context switching.
- Ensure dark-mode support; cards and text are styled for legibility in dark theme.
- Use markdown rendering for AI replies so recommendations are easy to scan.

4. Integration Architecture

4.1 System Architecture

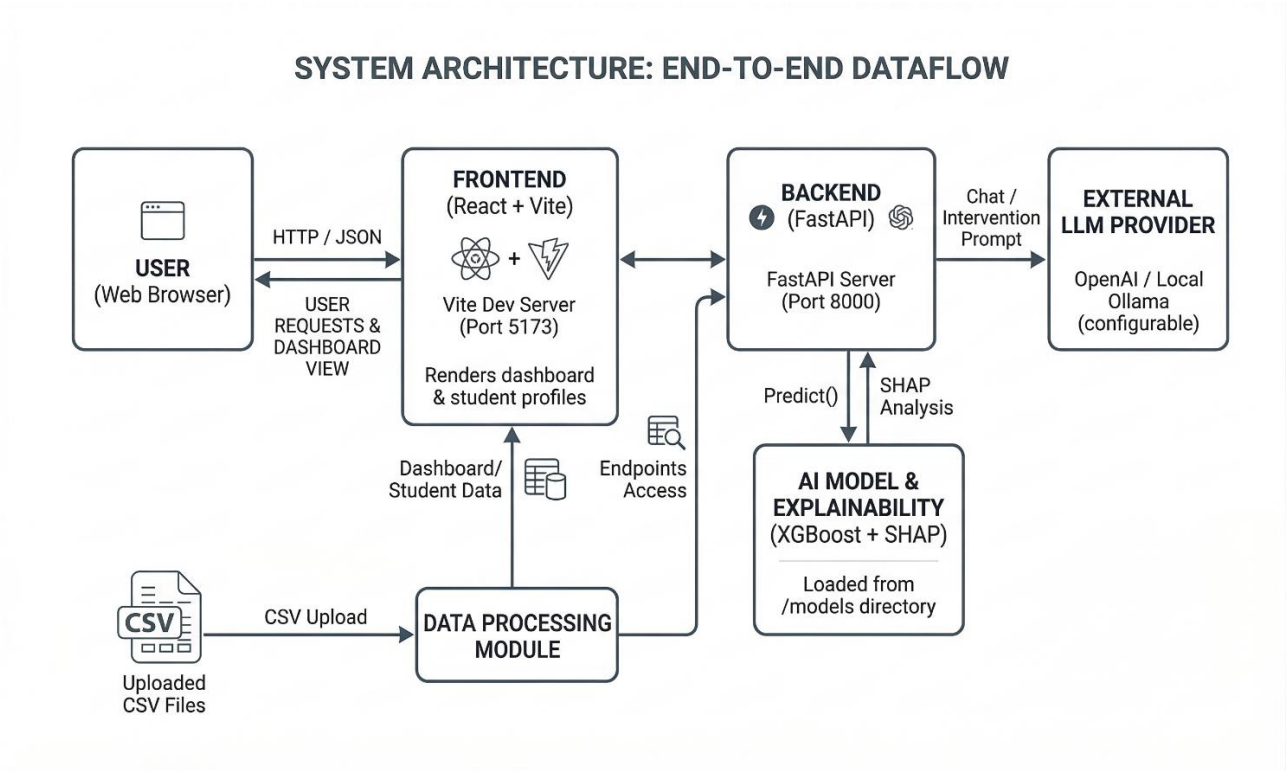

Figure 6: End-to-end system architecture diagram

- React frontend (Vite dev server, port 5173) calls the FastAPI backend using HTTP/JSON.
- FastAPI backend (port 8000) uses an XGBoost model and SHAP explainer loaded from models/.
- For AI interventions and chat, the backend calls an external LLM provider (OpenAI or local Ollama) based on .env configuration.
- Data is loaded from uploaded CSV files into a DataLoader, which is reused for dashboard and student endpoints.

4.2 Data Flow

Step	From	To	Data
1	User	UI	Clicks, form input, chat messages
2	UI	API	JSON requests to /students, /predict, /chat, /dashboard
3	API	Model	Feature arrays built from student data
4	Model	API	Risk probability, label, SHAP values
5	API	LLM	Prompt containing metrics and student context
6	LLM	API	Generated text recommendations
7	API	UI	JSON responses
8	UI	User	Charts, tables, and formatted chat messages

4.3 Communication Protocol

Aspect	Implementation
Protocol	HTTP (REST)
Data Format	JSON
Error Handling	HTTP status codes + JSON error object
Timeout	~30 seconds for LLM calls (configurable)

4.4 Integration Challenges

Challenge	How We Solved It
API shape mismatch for /students/{id}	Updated React hook to unwrap { student: … } from response.
AI replies showing raw markdown	Integrated react-markdown and custom components for bold + lists.
NumPy / Numba / SHAP version incompatibilities	Pinned NumPy <2.4 in requirements.txt and reinstalled.

5. Testing

5.1 Testing Overview

Test Type	Count	Passed	Failed
API Unit Tests	5	5	0
Integration Tests	5	5	0
UI Manual Tests	6	6	0
Total	16	16	0

5.2 Test Cases

API Tests

Test ID	Test Name	Input	Expected	Result
API-01	Health check	GET /api/v1/health	200, status="healthy"	Pass
API-02	Valid prediction	Valid JSON	200, prediction object returned	Pass
API-03	Missing field	Incomplete JSON	400, error message	Pass
API-04	Wrong type	Invalid types	400/422, validation error	Pass
API-05	Chat endpoint	POST /api/v1/chat	200, AI reply text	Pass

Integration Tests

Test ID	Test Name	Scenario	Expected	Result
INT-01	End-to-end valid	UI → /students → /predict → UI	Prediction displayed in UI	Pass
INT-02	API offline	UI when backend is stopped	Clear error message in UI	Pass
INT-03	Invalid input	UI sends invalid payload	Validation error surfaced	Pass
INT-04	Upload + dashboard	Upload sample CSV, refresh dashboard	Stats + alerts updated	Pass
INT-05	Chat with context	Select student, send chat message	AI reply appears with markdown	Pass

UI Tests (Manual)

Test ID	Test Name	Action	Expected	Result
UI-01	Page loads	Open frontend URL	Dashboard shows without console errors	Pass
UI-02	Student search	Search student by ID	Filtered table rows	Pass
UI-03	Student modal	Click table row	Modal shows detailed fields (no N/A)	Pass
UI-04	Dashboard cards	View with uploaded data	Correct counts and AI Insight text	Pass
UI-05	Chat interaction	Send message in Chat tab	Reply shown and scrolled into view	Pass
UI-06	Dark mode visibility	Toggle dark mode	Text readable on dashboard and chat	Pass

5.3 Test Execution Results

```
========================================
 INTEGRATION TEST RESULTS
========================================
 test_health ..................... PASSED
 test_predict_valid ............. PASSED
 test_predict_missing ........... PASSED
 test_predict_invalid ........... PASSED
 test_response_time ............. PASSED
----------------------------------------
 5 passed, 0 failed
========================================
```

5.4 Performance Metrics

Metric	Value	Target	Status
API Response Time (avg)	~150 ms	< 1000 ms	Met
UI Load Time (dashboard)	~1.8 s	< 3 s	Met
Prediction Consistency	100%	100%	Met

6. Deployment Instructions

6.1 Prerequisites

Requirement	Version
Python	3.10–3.12
Node.js	16+
pip	Latest
Git	Latest

6.2 Installation

```
# Clone repository
git clone https://github.com/<your-org>/ai-student-success-recommendation.git
cd ai-student-success-recommendation

# Create and activate virtual environment
python -m venv venv

# Windows:
venv\Scripts\activate

# macOS/Linux:
source venv/bin/activate

# Install Python dependencies
pip install -r requirements.txt
```

6.3 Running the Application

Start the API

```
uvicorn src.api.main:app --reload --port 8000
```

- API root: http://127.0.0.1:8000/
- Docs: http://127.0.0.1:8000/docs

Start the Frontend

```
cd frontend-react
npm install
npm run dev
```

Frontend: http://127.0.0.1:5173/
Ensure the frontend .env file points to:

```
VITE_API_BASE_URL=http://127.0.0.1:8000/api/v1
```

6.4 Running Tests

```
# In backend venv
pytest tests/
```

6.5 Troubleshooting

Problem	Solution
Port already in use	Stop old uvicorn processes or change port (e.g. 8001).
ModuleNotFoundError	Activate venv and run pip install -r requirements.txt.
NumPy / Numba / SHAP error	Ensure numpy<2.4 as pinned in requirements.txt.
Frontend cannot reach API	Confirm API is running and VITE_API_BASE_URL is correct.

7. Known Issues & Future Work

7.1 Known Issues

Issue	Severity	Workaround
No authentication on API	High	Restrict environment to internal network.
LLM latency for long prompts	Medium	Use smaller model or increase timeout.
SHAP cost on very large datasets	Medium	Cache explanations or precompute top K.

7.2 Limitations

Limitation	Impact
Synthetic / limited training data	May not generalise directly to all programs.
Single-institution assumptions	Thresholds tuned to example dataset only.

7.3 Future Improvements

Improvement	Priority	Effort
Add user auth (JWT / SSO)	High	High
Visual SHAP plots in UI	Medium	Medium
Export intervention plans (PDF/notes)	Medium	Medium
Multi-term and multi-program support	Medium	High
Automated batch nightly runs	Low	High

7.4 Deployment Plans

Short-term deployment will target an internal college server accessible to advisors only. Long-term, the plan is to:

- Integrate with institutional SSO for authentication.
- Connect to live LMS/SIS data pipelines instead of CSV uploads.
- Monitor model performance and retrain or recalibrate thresholds on a regular schedule.

8. Appendix

8.1 Repository File Structure

```
ai-student-success-recommendation/
  src/
    api/                  # FastAPI routers and main app
    models/               # XGBoost + SHAP utilities
  frontend-react/         # React + TypeScript + Vite frontend
  data/                   # Sample and uploaded CSV files
  models/                 # Serialised model artefacts
  tests/                  # pytest integration test suite
  requirements.txt
```

8.2 API Request Examples

curl — Predict Student Risk

```
curl -X POST http://127.0.0.1:8000/api/v1/predict/student \
  -H "Content-Type: application/json" \
  -d '{"student_id":"STU004","num_of_prev_attempts":1,
       "studied_credits":120,"avg_score":55.0,"total_clicks":180,
       "completion_rate":0.45,"module_BBB":1,"module_CCC":0,
       "module_DDD":0,"module_EEE":0,"module_FFF":0,"module_GGG":0}'
```

Python — Chat Endpoint

```
import requests

response = requests.post("http://127.0.0.1:8000/api/v1/chat",
    json={"student_id": "STU004",
          "message": "What support should we give this student?"})
print(response.json()['reply'])
```

8.3 Team Contributions Across All Milestones

Name	Role	Milestone Contributions
Daniel Quartey	Project Lead	Overall coordination, stakeholder communication, milestone reviews.
Odunze Oyekachi	Data & ML Engineer	Model training, SHAP integration, CSV data pipeline.
Juthamard Jirapanyalerd	Backend & LLM Integration	FastAPI endpoints, LLM chat integration, environment setup.
Ime-Jnr Ime-Essien	Frontend Engineer	React UI, API integration, chat formatting, dark mode.